

Android Application Security Audit Report

Target Application Vulnerability & Static Code Assessment

ASSESSMENT DATE
September 26, 2026
MASVS v2 Verified

Package: `com.onkar.androidtest` • Version `1.0`

OVERALL RISK RATING

LOW RISK

Minimal exposure footprint

HIGH SEVERITY ITEMS

1

allowBackup="true" (CWE-926)

MASVS SCORE

85 / 100

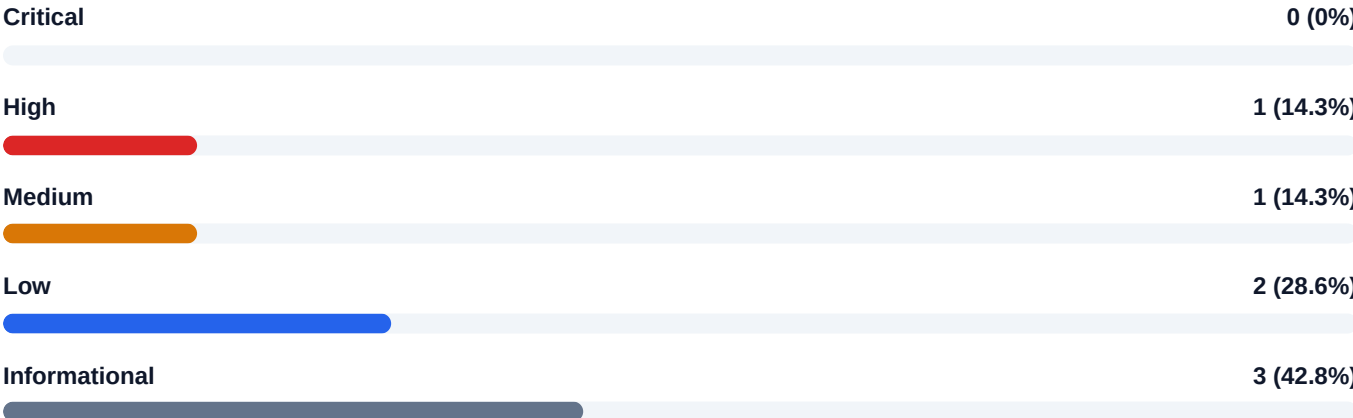
Grade B Compliance Level

TOTAL FINDINGS

7

1 High • 1 Med • 2 Low • 3 Info

Severity Distribution



Target Specification & Environment

Application APK	app-release.apk (7.9 MB)
SHA-256 Digest	9ff7c8ba8667...065ad9eb
Framework Stack	Kotlin + Jetpack Compose
Obfuscation	R8 (Resources Confusion Active)
Native Binaries	None (Pure DEX Bytecode)
Permissions	0 Dangerous, 1 Signature-only

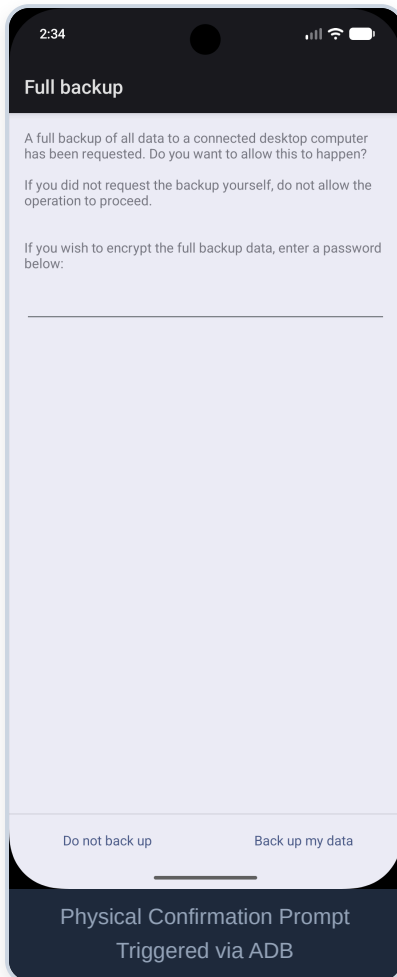
[HIGH-001] Data Extraction via ADB Backup (allowBackup="true")

SEVERITY: HIGH

CVSS: 5.9

CWE-926

OWASP: MASVS-STORAGE-1



HOW WE FOUND IT

Static manifest auditing with APKTool identified `android:allowBackup="true"` declared on line 4 of `AndroidManifest.xml`. We verified physical exploitability over USB debugging by invoking the backup bridge:

```
adb backup -f backup.ab com.onkar.androidtest
```

The connected phone immediately intercepted the call and presented the full-screen prompt shown on the left: "Full backup — A full backup of all data to a connected desktop computer has been requested..."

TECHNICAL EXPLANATION & ATTACK IMPACT

When `android:allowBackup` is set to `true`, any physical workstation with developer access can trigger an ad-hoc serial snapshot of the application's sandboxed storage directory (`/data/data/com.onkar.androidtest/`) without requiring root privileges.

```
# Attack Exfiltration Sequence:
dd if=backup.ab bs=24 skip=1 | zlib-flate -uncompress > backup.tar
tar -xf backup.tar
# Local SQLite DBs, Shared Preferences, and tokens are extracted.
```

Even though the initial release does not store sensitive credentials, leaving this enabled creates immediate data leak vulnerability the moment tokens or sensitive cache files are added.

WAY TO FIX IT (REMEDIATION)

Explicitly disable ADB serialization within `AndroidManifest.xml`:

```
<application
  android:allowBackup="false"
  ... >
```

If automated Google Cloud user migration is strictly necessary, supply a scoped filtering rule:

```
<!-- res/xml/backup_rules.xml -->
<full-backup-content>
  <exclude domain="database" path="."/>
  <exclude domain="sharedpref" path="."/>
  <exclude domain="file" path="."/>
</full-backup-content>
```

[MED-001] Exported ProfileInstaller with System Permission

SEVERITY: MEDIUM

CVSS: 4.3

CWE-926

OWASP: MASVS-PLATFORM-2

HOW WE FOUND IT

Manifest disassembly exposed an exported broadcast listener protected by a signature-level system permission:

```
<receiver android:name="androidx.profileinstaller.ProfileInstallerReceiver"
  android:exported="true"
  android:permission="android.permission.DUMP">
  <intent-filter>
    <action android:name="androidx.profileinstaller.action.INSTALL_PROFILE"/>
  </intent-filter>
</receiver>
```

We confirmed component accessibility via ADB broadcast invocation:

```
adb shell am broadcast -a androidx.profileinstaller.action.INSTALL_PROFILE \
  -n com.onkar.androidtest/androidx.profileinstaller.ProfileInstallerReceiver
# Broadcast completed: result=0
```

TECHNICAL EXPLANATION & ATTACK IMPACT

This receiver originates from Google's AndroidX ProfileInstaller library, intended for AOT compilation optimization benchmarking. While protected by `android.permission.DUMP` (preventing normal user apps from triggering it), it remains an open IPC target for shell or privileged device processes, presenting an unnecessary attack surface for Denial of Service (DoS) or benchmark manipulation.

WAY TO FIX IT (REMEDIATION)

If production benchmarking is not required, strip the dependency or suppress it through manifest merger:

```
<!-- AndroidManifest.xml -->
<receiver
  android:name="androidx.profileinstaller.ProfileInstallerReceiver"
  tools:node="remove" />
```

[LOW-001] Resources Confusion (R8 Artifact)

SEVERITY: LOW

CVSS: 2.1

CWE-693

🔍 HOW WE FOUND IT

Running APKiD against the package identified a known bytecode manipulator signature:

```
apkid app-release.apk
# |-> manipulator : Resources Confusion (classes.dex)
```

💡 TECHNICAL EXPLANATION

This is an aggressive R8 optimization technique that scrambles resource tables and IDs to reduce binary size. While it slows down reverse engineering, it can trigger resource loading crashes or collisions on custom Android ROMs (OneUI, HyperOS).

🔧 WAY TO FIX IT

Verify UI rendering across real hardware. If crashes occur, add resource retention exceptions inside `proguard-rules.pro`.

[LOW-002] Missing Network Security Config

SEVERITY: LOW

CVSS: 3.7

CWE-319

🔍 HOW WE FOUND IT

Decompiled directory inspection verified that `res/xml/network_security_config.xml` is absent, and the manifest contains no configuration declaration.

💡 TECHNICAL EXPLANATION

Although the APK currently declares no `android.permission.INTERNET`, adding API connectivity in future versions without this file leaves communication open to cleartext HTTP fallback or rogue root CAs on pre-Android 9 devices.

🔧 WAY TO FIX IT

Add `res/xml/network_security_config.xml`:

```
<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors><certificates
src="system"/></trust-anchors>
  </base-config>
</network-security-config>
```

Reference it in `AndroidManifest.xml`:

```
<application android:networkSecurityC
onfig="@xml/network_security_config"
... >
```

Informational & Framework Observations

[INFO-001] Anti-VM Logic

FRAMEWORK

Found: APKiD flagged `Build.MANUFACTURER` in `classes2.dex`.

Explanation: Located at `androidx.core.view.DisplayCompat:85`. It detects Sony BRAVIA 4K screens for display scaling. Completely benign framework code.

Fix: No action required.

[INFO-002] Stripped R8 Marker

OPTIMIZER

Found: APKiD flagged DEX as *"r8 without marker"*.

Explanation: Normal minification for release builds where compilation annotations are stripped to compress binary footprint.

Fix: No action required.

[INFO-003] Certificate Validity

27 YEARS

Found: `keytool` reported signing validity until **February 11, 2054**.

Explanation: Long lifespans allow smooth updates, but 27-year validity extends the window of risk if private keystores are compromised.

Fix: Standardize on 10–15 year certificate expiry for production releases.

OWASP MASVS v2 Compliance Matrix

Domain	Control ID	Evaluation	Technical Observation
Data Storage	MASVS-STORAGE-1	FAIL	<code>android:allowBackup="true"</code> enables full private data extraction via ADB.
Platform Interaction	MASVS-PLATFORM-2	PASS	Exported ProfileInstallReceiver is protected by system-level <code>DUMP</code> permission.
Network Communication	MASVS-NETWORK-1	PASS	No Internet permissions requested; explicit network config recommended for future.
Code Resilience	MASVS-RESILIENCE-2	PASS	R8 minification and resource obfuscation actively applied across all DEX classes.
User Privacy	MASVS-PRIVACY-1	PASS	No telemetry trackers, advertising IDs, or PII harvesting present in codebase.

Prioritized Remediation Roadmap

Priority	Recommended Action	Target File	Security Impact
P1	Set <code>android:allowBackup="false"</code>	<code>AndroidManifest.xml</code>	Completely blocks unauthorized adb data extraction.
P2	Enforce <code>network_security_config.xml</code>	<code>res/xml/</code> & Manifest	Guarantees TLS-only transmission if API features are introduced.
P3	Prune <code>ProfileInstallReceiver</code>	<code>build.gradle.kts</code>	Reduces IPC attack surface by stripping unnecessary test hooks.
P4	Recalibrate Signing Key Duration	Keystore Generation	Adopts standard 10–15 year certificate rotation practice.